

Reviewer Pack — Dyadic Spectrum Entropy Invariants for Resolution Proof Dept...

Entry bdbaaf352ae0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 17:06:59 UTC

Dyadic Spectrum Entropy Invariants for Resolution Proof Depth

Entry ID: bdbaaf352ae0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 17:06:59 UTC

1. Conjecture

Field A (mathematical branch): Dyadic Harmonic Analysis (Signal processing) **Field B** (complexity object): Resolution proof depth

Statement:

For every resolution proof tree with m clauses and n variables, the entropy of its dyadic spectrum $H_d(PT)$ is at most $O(\log(m+n))$. Specifically, for a fixed constant c , $|H_d(PT)| \leq c * \log(m+n)$. Moreover, if $H_d(PT) = \Theta(\log(m+n))$, then the resolution proof depth is also $\Theta(\log(m+n))$.

Rationale (proposer's reasoning):

Dyadic harmonic analysis provides a framework to analyze the frequency content of signals. In this conjecture, we leverage dyadic spectrum entropy to provide bounds on the resolution proof depth. The structure revealed by the spectrum might expose underlying patterns in the proof tree that influence its complexity.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9fd37d3d0115f97b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a CNF formula with m clauses and n variables, $|H_d(PT)|$ must be $\leq c * \log(m+n)$ to support the conjecture, where c is a fixed constant. Falsification occurs if any seed produces $|H_d(PT)| > c * \log(m+n)$, or if the correlation between $H_d(PT)$ and resolution proof depth is not significant at $p < 0.05$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -dyadic harmonic analysis AND resolution proof depth - resolution proof depth INVARIANTS dyadic spectrum entropy - entropy of dyadic spectrum Resolution proof tree complexity

Top relevant hits considered: - [http://arxiv.org/abs/1812.00850v1] Dyadic harmonic analysis and weighted inequalities: the sparse revolution - [http://arxiv.org/abs/1604.02789v1] An alternative proof of a sharp generalization of an integral inequality for the dyadic maximal operator and application - [http://arxiv.org/abs/astro-ph/0208243v7] Measurement of the Flux of Ultra-high Energy Cosmic Rays from Monocular Observations by the High Resolution Fly’s Eye Exp - [http://arxiv.org/abs/1807.02622v2] Rényi Entropy Power Inequalities via Normal Transport and Rotation - [http://arxiv.org/abs/2112.13763v3] Yet Another Proof of the Joint Convexity of Relative Entropy - [http://arxiv.org/abs/1809.06500v3] Range entropy: A bridge between signal complexity and self-similarity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        cnf = []
        for _ in range(m):
            clause = [random.randint(1, n), -random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def resolution_proof_depth(cnf):
        clauses = set(tuple(sorted(c)) for c in cnf)
        depth = 0
        while True:
            new_clauses = []
            for c1 in clauses:
                for c2 in clauses:
                    if len(set(c1) & set(c2)) == 2:
                        lit = list(set(c1) ^ set(c2))[0]
                        new_clause = [lit] + [l for l in c1 if l != -lit]
                        new_clauses.append(new_clause)
```

```

        if not new_clauses:
            break
        clauses.update(tuple(sorted(c)) for c in new_clauses)
        depth += 1
    return depth

def dyadic_spectrum(clause):
    n = len(clause)
    spectrum = [0] * (n + 1)
    for i in range(n):
        spectrum[i % (n + 1)] += abs(clause[i])
    return spectrum

def entropy(spectrum):
    total = sum(spectrum)
    if total == 0:
        return 0
    return -sum(x / total * math.log2(x / total) for x in spectrum if x > 0)

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    m = random.randint(1, n * 2)
    cnf = generate_cnf(n, m)
    depth = resolution_proof_depth(cnf)
    spectrum = dyadic_spectrum(cnf)
    H_d = entropy(spectrum)
    results.append({
        "n": n,
        "m": m,
        "depth": depth,
        "H_d": H_d
    })

mean_H_d = sum(r["H_d"] for r in results) / len(results)
std_H_d = math.sqrt(sum((r["H_d"] - mean_H_d) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["H_d"] <= 0.5 * math.log2(r["m"] + r["n"])) / len(results)

conjecture_holds = support_fraction >= 0.8
counterexample = "" if conjecture_holds else f"H_d = {mean_H_d}, m+n = {sum(r['m'] + r['n'] for r in results)}"

return {
    "metric_name": "H_d",
    "metric_value": mean_H_d,
    "instances_tested": len(results),
    "n_max": max(r["n"] for r in results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

print(f"TRIAL: {result}")
results.append(result)

mean_H_d = sum(r["metric_value"] for r in results) / len(results)
std_H_d = math.sqrt(sum((r["metric_value"] - mean_H_d) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_H_d} std={std_H_d} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_54fbdd58.py", line 94, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_54fbdd58.py", line 63, in run_trial
    depth = resolution_proof_depth(cnf)
            ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_54fbdd58.py", line 36, in resolution_proof_depth
    lit = list(set(c1) ^ set(c2))[0]
           ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's validity. | next: Investigate and fix the crash in the test code to proceed with the evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	24268
2	preregistration	ollama_remote	glm4:latest	0	0	9878
3	novelty	ollama_remote	glm4:latest	0	0	8411
4	novelty	ollama_remote	glm4:latest	0	0	9922
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16872
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10467
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12638
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	51436
9	judge	ollama_remote	glm4:latest	0	0	79120

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 223012 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/bdbaaf352ae0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/bdbaaf352ae0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/bdbaaf352ae0.tar.gz` (if generated)